

Objective

We'll build a simple ASP.NET Core Web API that:

1. Connects securely to Azure Key Vault
 2. Retrieves a **database connection string** secret
 3. Uses **Azure.Identity** for authentication (no credentials in code)
-

Step 1: Prerequisites

You'll need:

-  Azure subscription
 -  Created **Azure Key Vault**
 -  Added a secret (e.g., DbConnectionString)
 -  Created a **Web App** in Azure with **System-assigned Managed Identity** enabled
-

Step 2: .NET Web API Setup

Create a new project:

```
dotnet new webapi -n KeyVaultDemoAPI
```

```
cd KeyVaultDemoAPI
```

Add the required NuGet packages:

```
dotnet add package Azure.Identity
```

```
dotnet add package Azure.Security.KeyVault.Secrets
```

Step 3: Folder & Code Structure

KeyVaultDemoAPI/

├── Controllers/

| └── SecretsController.cs

|— Program.cs

|— appsettings.json

Step 4: Program.cs

Here's the main configuration file:

```
using Azure.Identity;
```

```
using Azure.Security.KeyVault.Secrets;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
//  Configure Key Vault integration
```

```
string keyVaultUrl = builder.Configuration["KeyVaultUrl"];
```

```
builder.Configuration.AddAzureKeyVault(new Uri(keyVaultUrl), new DefaultAzureCredential());
```

```
builder.Services.AddControllers();
```

```
var app = builder.Build();
```

```
app.MapControllers();
```

```
app.Run();
```

◆ Explanation:

- AddAzureKeyVault() loads secrets directly into your app configuration.
 - DefaultAzureCredential() automatically detects:
 - Managed Identity (when deployed in Azure)
 - Visual Studio / CLI credentials (when running locally)
-

Step 5: Add appsettings.json

```
{  
    "KeyVaultUrl": "https://<your-keyvault-name>.vault.azure.net/"  
}
```

✅ Replace <your-keyvault-name> with your Key Vault's name.

Step 6: Create a Controller

Create a new file:

 Controllers/SecretsController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using Azure.Identity;
```

```
using Azure.Security.KeyVault.Secrets;
```

```
namespace KeyVaultDemoAPI.Controllers
```

```
{
```

```
    [ApiController]
```

```
    [Route("api/[controller]")]
```

```
    public class SecretsController : ControllerBase
```

```
    {
```

```
        private readonly SecretClient _secretClient;
```

```
        public SecretsController(IConfiguration configuration)
```

```
        {
```

```
            var keyVaultUrl = configuration["KeyVaultUrl"];
```

```
            _secretClient = new SecretClient(new Uri(keyVaultUrl), new DefaultAzureCredential());
```

```
        }
```

```
[HttpGet("dbconnection")]
public async Task<IActionResult> GetDbConnectionAsync()
{
    try
    {
        KeyVaultSecret secret = await _secretClient.GetSecretAsync("DbConnectionString");
        return Ok(new { ConnectionString = secret.Value });
    }
    catch (Exception ex)
    {
        return BadRequest(new { Error = ex.Message });
    }
}
}
```

◆ **Explanation:**

- Connects to the Key Vault using **DefaultAzureCredential**
- Retrieves the secret named **DbConnectionString**
- Returns it as API response (for demo purposes)

Step 7: Run Locally

Before running locally, make sure you have access to the Key Vault:

Option 1: Using Your Azure Login

Login to Azure CLI:

```
az login
```

Run your app:

dotnet run

Then test the API:

GET https://localhost:5001/api/secrets/dbconnection

✅ You should see your connection string as JSON.

🧠 Step 8: Deploy to Azure App Service

1. Publish your app:
2. dotnet publish -c Release
3. Deploy to Azure App Service (via VS, GitHub Actions, or CLI)
4. Enable **System-assigned Managed Identity**:
 - In Portal → App Service → **Identity** → **System assigned** → **On**
5. In your **Key Vault**:
 - Go to **Access control (IAM)** or **Access policies**
 - Add your Web App's Managed Identity
 - Grant **Get and List** permissions for Secrets

✅ Once deployed, your Web App can securely access secrets from Key Vault.

🧠 Step 9: How Authentication Works Behind the Scenes

- When running locally → uses your Azure CLI or VS credentials
 - When running in Azure → uses **Managed Identity** automatically
 - No need to store passwords, keys, or tokens in code
 - Azure handles token retrieval and refresh securely
-

🧱 Step 10: Example Output

Calling:

GET https://<your-app>.azurewebsites.net/api/secrets/dbconnection

Response:

```
{  
  "connectionString": "Server=tcp:mydb.database.windows.net;Database=DemoDB;User  
Id=admin;Password=P@ssw0rd;"  
}
```

Step 11: Best Practices

- ✓ Never log secrets or return them in production responses
 - ✓ Use Key Vault references in App Service settings if you prefer configuration-only approach
 - ✓ Apply RBAC for principle of least privilege
 - ✓ Enable soft delete and purge protection in Key Vault
 - ✓ Rotate secrets periodically
-

Summary

Step Description

- 1** Created Azure Key Vault & added a secret
- 2** Created ASP.NET Core Web API
- 3** Integrated Key Vault using DefaultAzureCredential()
- 4** Accessed the secret securely (no credentials in code)
- 5** Deployed with Managed Identity